

Shell Prompt

The prompt, \$, which is called the **command prompt**, is issued by the shell. While the prompt is displayed, you can type a command.

Shell reads your input after you press **Enter**. It determines the command you want executed by looking at the first word of your input. A word is an unbroken set of characters. Spaces and tabs separate words.

Following is a simple example of the **date** command, which displays the current date and time –

```
$date  
Thu Jun 25 08:30:19 MST 2009
```

You can customize your command prompt using the environment variable PS1 explained in the Environment tutorial.

Shell Types

In Unix, there are two major types of shells –

- **Bourne shell** – If you are using a Bourne-type shell, the \$ character is the default prompt.
- **C shell** – If you are using a C-type shell, the % character is the default prompt.

The Bourne Shell has the following subcategories –

- Bourne shell (sh)
- Korn shell (ksh)
- Bourne Again shell (bash)
- POSIX shell (sh)

The different C-type shells follow –

- C shell (csh)
- TENEX/TOPS C shell (tcsh)

The original Unix shell was written in the mid-1970s by Stephen R. Bourne while he was at the AT&T Bell Labs in New Jersey.

Bourne shell was the first shell to appear on Unix systems, thus it is referred to as "the shell".

Bourne shell is usually installed as **/bin/sh** on most versions of Unix. For this reason, it is the shell of choice for writing scripts that can be used on different versions of Unix.

In this chapter, we are going to cover most of the Shell concepts that are based on the Bourne Shell.

Shell Scripts

The basic concept of a shell script is a list of commands, which are listed in the order of execution. A good shell script will have comments, preceded by # sign, describing the steps.

There are conditional tests, such as value A is greater than value B, loops allowing us to go through massive amounts of data, files to read and store data, and variables to read and store data, and the script may include functions.

We are going to write many scripts in the next sections. It would be a simple text file in which we would put all our commands and several other required constructs that tell the shell environment what to do and when to do it.

Shell scripts and functions are both interpreted. This means they are not compiled.

Example Script

Assume we create a **test.sh** script. Note all the scripts would have the **.sh** extension. Before you add anything else to your script, you need to alert the system that a shell script is being started. This is done using the **shebang** construct. For example –

```
#!/bin/sh
```

This tells the system that the commands that follow are to be executed by the Bourne shell. *It's called a shebang because the # symbol is called a hash, and the ! symbol is called a bang.*

To create a script containing these commands, you put the shebang line first and then add the commands –

```
#!/bin/bash
```

```
pwd
```

```
ls
```

```
Variable
```

A variable is nothing more than a pointer to the actual data. The shell enables you to create, assign, and delete variables.

Variable Names

The name of a variable can contain only letters (a to z or A to Z), numbers (0 to 9) or the underscore character (_).

By convention, Unix shell variables will have their names in UPPERCASE.

The following examples are valid variable names –

```
_ALI  
TOKEN_A  
VAR_1  
VAR_2
```

Following are the examples of invalid variable names –

```
2_VAR  
-VARIABLE  
VAR1-VAR2  
VAR_A!
```

The reason you cannot use other characters such as !, *, or - is that these characters have a special meaning for the shell.

Defining Variables

Variables are defined as follows –

variable_name=variable_value

For example –

```
NAME="Zara Ali"
```

The above example defines the variable NAME and assigns the value "Zara Ali" to it. Variables of this type are called **scalar variables**. A scalar variable can hold only one value at a time.

Shell enables you to store any value you want in a variable. For example –

```
VAR1="Zara Ali"
```

```
VAR2=100
```

Accessing Values

To access the value stored in a variable, prefix its name with the dollar sign (\$) –

For example, the following script will access the value of defined variable NAME and print it on STDOUT –

```
#!/bin/sh  
  
NAME="Zara Ali"  
echo $NAME
```

The above script will produce the following value –

Zara Ali

Variable Types

When a shell is running, three main types of variables are present –

- **Local Variables** – A local variable is a variable that is present within the current instance of the shell. It is not available to programs that are started by the shell. They are set at the command prompt.
- **Environment Variables** – An environment variable is available to any child process of the shell. Some programs need environment variables in order to function correctly. Usually, a shell script defines only those environment variables that are needed by the programs that it runs.
- **Shell Variables** – A shell variable is a special variable that is set by the shell and is required by the shell in order to function correctly. Some of these variables are environment variables whereas others are local variables.

Command-Line Arguments or special parameters used by shell/ command line

we understood how to be careful when we use certain non alphanumeric characters in variable names. This is because those characters are used in the names of special Unix variables. These variables are reserved for specific functions.

- For example, the \$ character represents the process ID number, or PID, of the current shell –

```
$echo $$  
$
```

- The above command writes the PID of the current shell –
- 29949
- The following table shows a number of **special variables or parameters** that you can use in your shell scripts –

Sr.No.	Variable & Description
1	\$0 The filename of the current script.
2	\$n or \$1,\$2,\$3.... These variables correspond to the arguments with which a script was invoked. Here n is a positive decimal number corresponding to the position of an argument (the first argument is \$1, the second argument is \$2, and so on). Or Positional parameters
3	\$# The number of arguments supplied to a script.
4	\$* All the arguments are double quoted. If a script receives two arguments, \$* is equivalent to \$1 \$2.
5	\$@ All the arguments are individually double quoted. If a script receives two arguments, @\$ is equivalent to \$1 \$2.
6	\$? The exit status of the last command executed.
7	\$\$ The process number of the current shell. For shell scripts, this is the process ID under which they are executing.
8	\$_ The process number of the last background command.

- Command-Line Arguments
 - The command-line arguments \$1, \$2, \$3, ...\$9 are positional parameters, with \$0 pointing to the actual command, program, shell script, or function and \$1, \$2, \$3, ...\$9 as the arguments to the command.
 - Following script uses various special variables related to the command line –
- **#!/bin/sh**

-
- `echo "File Name: $0"`
- `echo "First Parameter : $1"`
- `echo "Second Parameter : $2"`
- `echo "Quoted Values: $@"`
- `echo "Quoted Values: $*"`
- `echo "Total Number of Parameters : $#"`

- Here is a sample run for the above script –
- `./test.sh Zara Ali`
- File Name : `./test.sh`
- First Parameter : Zara
- Second Parameter : Ali
- Quoted Values: Zara Ali
- Quoted Values: Zara Ali
- Total Number of Parameters : 2
- Special Parameters `$*` and `$@`
- There are special parameters that allow accessing all the command-line arguments at once. `$*` and `$@` both will act the same unless they are enclosed in double quotes, `""`.
- Both the parameters specify the command-line arguments. However, the `"$"` special parameter takes the entire list as one argument with spaces between and the `"$@"` special parameter takes the entire list and separates it into separate arguments.
- We can write the shell script as shown below to process an unknown number of commandline arguments with either the `$*` or `$@` special parameters –

- `#!/bin/sh`
-
- `for TOKEN in $*`
- `do`
- `echo $TOKEN`
- `done`

- Here is a sample run for the above script –
- `./test.sh Zara Ali 10 Years Old`
- Zara
- Ali
- 10
- Years

Exit Status

- The `$?` variable represents the exit status of the previous command.
- Exit status is a numerical value returned by every command upon its completion. As a rule, most commands return an exit status of 0 if they were successful, and 1 if they were unsuccessful.
- Some commands return additional exit statuses for particular reasons. For example, some commands differentiate between kinds of errors and will return various exit values depending on the specific type of failure.
- Following is the example of successful command –
- `./test.sh Zara Ali`
- File Name : `./test.sh`
- First Parameter : Zara

- Second Parameter : Ali
- Quoted Values: Zara Ali
- Quoted Values: Zara Ali
- Total Number of Parameters : 2
- \$echo \$?
- 0
- \$

Array

Shell supports a different type of variable called an **array variable**. This can hold multiple values at the same time. Arrays provide a method of grouping a set of variables. Instead of creating a new name for each variable that is required, you can use a single array variable that stores all the other variables.

All the naming rules discussed for Shell Variables would be applicable while naming arrays.

Defining Array Values

The difference between an array variable and a scalar variable can be explained as follows.

Suppose you are trying to represent the names of various students as a set of variables. Each of the individual variables is a scalar variable as follows –

```
NAME01="Zara"
NAME02="Qadir"
NAME03="Mahnaz"
NAME04="Ayan"
NAME05="Daisy"
```

We can use a single array to store all the above mentioned names. Following is the simplest method of creating an array variable. This helps assign a value to one of its indices.

`array_name[index]=value`

Here *array_name* is the name of the array, *index* is the index of the item in the array that you want to set, and *value* is the value you want to set for that item.

As an example, the following commands –

```
NAME[0]="Zara"
NAME[1]="Qadir"
NAME[2]="Mahnaz"
NAME[3]="Ayan"
NAME[4]="Daisy"
```

If you are using the **ksh** shell, here is the syntax of array initialization –

`set -A array_name value1 value2 ... valuen`

If you are using the **bash** shell, here is the syntax of array initialization –

`array_name=(value1 ... valuen)`

Accessing Array Values

After you have set any array variable, you access it as follows –

`${array_name[index]}`

Here *array_name* is the name of the array, and *index* is the index of the value to be accessed. Following is an example to understand the concept –

```
#!/bin/sh

NAME[0]="Zara"
NAME[1]="Qadir"
NAME[2]="Mahnaz"
NAME[3]="Ayan"
NAME[4]="Daisy"
echo "First Index: ${NAME[0]}"
echo "Second Index: ${NAME[1]}"
```

The above example will generate the following result –

```
$/test.sh
First Index: Zara
Second Index: Qadir
```

You can access all the items in an array in one of the following ways –

```
${array_name[*]}
${array_name[@]}
```

Here **array_name** is the name of the array you are interested in. Following example will help you understand the concept –

```
#!/bin/sh

NAME[0]="Zara"
NAME[1]="Qadir"
NAME[2]="Mahnaz"
NAME[3]="Ayan"
NAME[4]="Daisy"
echo "First Method: ${NAME[*]}"
echo "Second Method: ${NAME[@]}"
```

The above example will generate the following result –

```
$/test.sh
First Method: Zara Qadir Mahnaz Ayan Daisy
Second Method: Zara Qadir Mahnaz Ayan Daisy
```

Operators

There are various operators supported by each shell. We will discuss in detail about Bourne shell (default shell) in this chapter.

We will now discuss the following operators –

- Arithmetic Operators
- Relational Operators
- Boolean Operators
- String Operators
- File Test Operators

Bourne shell didn't originally have any mechanism to perform simple arithmetic operations but it uses external programs, either **awk** or **expr**.

The following example shows how to add two numbers –

```
#!/bin/sh  
  
val=`expr 2 + 2`  
echo "Total value : $val"
```

The above script will generate the following result –

Total value : 4

The following points need to be considered while adding –

- There must be spaces between operators and expressions. For example, 2+2 is not correct; it should be written as 2 + 2.
- The complete expression should be enclosed between ‘ ` ’, called the backtick.

Arithmetic Operators

The following arithmetic operators are supported by Bourne Shell.

Assume variable **a** holds 10 and variable **b** holds 20 then –

[Show Examples](#)

Operator	Description	Example
+ (Addition)	Adds values on either side of the operator	`expr \$a + \$b` will give 30
- (Subtraction)	Subtracts right hand operand from left hand operand	`expr \$a - \$b` will give -10
* (Multiplication)	Multiplies values on either side of the operator	`expr \$a \* \$b` will give 200
/ (Division)	Divides left hand operand by right hand operand	`expr \$b / \$a` will give 2
% (Modulus)	Divides left hand operand by right hand operand and returns remainder	`expr \$b % \$a` will give 0
= (Assignment)	Assigns right operand in left operand	a = \$b would assign value of b into a
== (Equality)	Compares two numbers, if both are same then returns true.	[\$a == \$b] would return false.

!= (Not Equality)	Compares two numbers, if both are different then returns true.	[\$a != \$b] would return true.
-------------------	--	-----------------------------------

It is very important to understand that all the conditional expressions should be inside square braces with spaces around them, for example [\$a == \$b] is correct whereas, [\$a==\$b] is incorrect.

All the arithmetical calculations are done using long integers.

Relational Operators

Bourne Shell supports the following relational operators that are specific to numeric values. These operators do not work for string values unless their value is numeric.

For example, following operators will work to check a relation between 10 and 20 as well as in between "10" and "20" but not in between "ten" and "twenty".

Assume variable **a** holds 10 and variable **b** holds 20 then –

[Show Examples](#)

Operator	Description	Example
-eq	Checks if the value of two operands are equal or not; if yes, then the condition becomes true.	[\$a -eq \$b] is not true.
-ne	Checks if the value of two operands are equal or not; if values are not equal, then the condition becomes true.	[\$a -ne \$b] is true.
-gt	Checks if the value of left operand is greater than the value of right operand; if yes, then the condition becomes true.	[\$a -gt \$b] is not true.
-lt	Checks if the value of left operand is less than the value of right operand; if yes, then the condition becomes true.	[\$a -lt \$b] is true.
-ge	Checks if the value of left operand is greater than or equal to the value of right operand; if yes, then the condition becomes true.	[\$a -ge \$b] is not true.
-le	Checks if the value of left operand is less than or equal to the value of right operand; if yes, then the condition becomes true.	[\$a -le \$b] is true.

It is very important to understand that all the conditional expressions should be placed inside square braces with spaces around them. For example, [\$a <= \$b] is correct whereas, [\$a <= \$b] is incorrect.

Boolean Operators

The following Boolean operators are supported by the Bourne Shell.

Assume variable **a** holds 10 and variable **b** holds 20 then –

[Show Examples](#)

Operator	Description	Example
!	This is logical negation. This inverts a true condition into false and vice versa.	[! false] is true.
-o	This is logical OR . If one of the operands is true, then the condition becomes true.	[\$a -lt 20 -o \$b -gt 100] is true.
-a	This is logical AND . If both the operands are true, then the condition becomes true otherwise false.	[\$a -lt 20 -a \$b -gt 100] is false.

String Operators

The following string operators are supported by Bourne Shell.

Assume variable **a** holds "abc" and variable **b** holds "efg" then –

[Show Examples](#)

Operator	Description	Example
=	Checks if the value of two operands are equal or not; if yes, then the condition becomes true.	[\$a = \$b] is not true.
!=	Checks if the value of two operands are equal or not; if values are not equal then the condition becomes true.	[\$a != \$b] is true.
-z	Checks if the given string operand size is zero; if it is zero length, then it returns true.	[-z \$a] is not true.
-n	Checks if the given string operand size is non-zero; if it is nonzero length, then it returns true.	[-n \$a] is not false.
str	Checks if str is not the empty string; if it is empty, then it returns false.	[\$a] is not false.

File Test Operators

We have a few operators that can be used to test various properties associated with a Unix file.

Assume a variable **file** holds an existing file name "test" the size of which is 100 bytes and has **read**, **write** and **execute** permission on –

[Show Examples](#)

Operator	Description	Example
-b file	Checks if file is a block special file; if yes, then the condition becomes true.	[-b \$file] is false.
-c file	Checks if file is a character special file; if yes, then the condition becomes true.	[-c \$file] is false.
-d file	Checks if file is a directory; if yes, then the condition becomes true.	[-d \$file] is not true.
-f file	Checks if file is an ordinary file as opposed to a directory or special file; if yes, then the condition becomes true.	[-f \$file] is true.
-g file	Checks if file has its set group ID (SGID) bit set; if yes, then the condition becomes true.	[-g \$file] is false.
-k file	Checks if file has its sticky bit set; if yes, then the condition becomes true.	[-k \$file] is false.
-p file	Checks if file is a named pipe; if yes, then the condition becomes true.	[-p \$file] is false.
-t file	Checks if file descriptor is open and associated with a terminal; if yes, then the condition becomes true.	[-t \$file] is false.
-u file	Checks if file has its Set User ID (SUID) bit set; if yes, then the condition becomes true.	[-u \$file] is false.
-r file	Checks if file is readable; if yes, then the condition becomes true.	[-r \$file] is true.
-w file	Checks if file is writable; if yes, then the condition becomes true.	[-w \$file] is true.
-x file	Checks if file is executable; if yes, then the condition becomes true.	[-x \$file] is true.
-s file	Checks if file has size greater than 0; if yes, then condition becomes true.	[-s \$file] is true.

-e file	Checks if file exists; is true even if file is a directory but exists.	[-e \$file] is true.
----------------	--	------------------------

Loop statements

Unix Shell supports conditional statements which are used to perform different actions based on different conditions. We will now understand two decision-making statements here –

- The **if...else** statement
- The **case...esac** statement

The if...else statements

If else statements are useful decision-making statements which can be used to select an option from a given set of options.

Unix Shell supports following forms of **if...else** statement –

- if...fi statement
- if...else...fi statement
- if...elif...else...fi statement

Most of the if statements check relations using relational operators discussed in the previous chapter.

The case...esac Statement

You can use multiple **if...elif** statements to perform a multiway branch. However, this is not always the best solution, especially when all of the branches depend on the value of a single variable.

Unix Shell supports **case...esac** statement which handles exactly this situation, and it does so more efficiently than repeated **if...elif** statements.

There is only one form of **case...esac** statement which has been described in detail here –

- case...esac statement

The **case...esac** statement in the Unix shell is very similar to the **switch...case** statement we have in other programming languages like **C** or **C++** and **PERL**, etc.

- [The while loop](#)
- [The for loop](#)
- [The until loop](#)
- [The select loop](#)

You will use different loops based on the situation. For example, the **while** loop executes the given commands until the given condition remains true; the **until** loop executes until a given condition becomes true.

Once you have good programming practice you will gain the expertise and thereby, start using appropriate loop based on the situation. Here, **while** and **for** loops are available in most of the other programming languages like **C**, **C++** and **PERL**, etc.

Nesting Loops

All the loops support nesting concept which means you can put one loop inside another similar one or different loops. This nesting can go up to unlimited number of times based on your requirement.

Here is an example of nesting **while** loop. The other loops can be nested based on the programming requirement in a similar way –

Nesting while Loops

It is possible to use a while loop as part of the body of another while loop.

Syntax

```
while command1 ; # this is loop1, the outer loop
do
    Statement(s) to be executed if command1 is true

    while command2 ; # this is loop2, the inner loop
    do
        Statement(s) to be executed if command2 is true
    done

    Statement(s) to be executed if command1 is true
done
```

Example

Here is a simple example of loop nesting. Let's add another countdown loop inside the loop that you used to count to nine –

```
#!/bin/sh

a=0
while [ "$a" -lt 10 ] # this is loop1
do
    b="$a"
    while [ "$b" -ge 0 ] # this is loop2
    do
        echo -n "$b "
        b=`expr $b - 1`
    done
    echo
    a=`expr $a + 1`
done
```

This will produce the following result. It is important to note how **echo -n** works here. Here **-n** option lets echo avoid printing a new line character.

```
0
1 0
2 1 0
3 2 1 0
4 3 2 1 0
5 4 3 2 1 0
6 5 4 3 2 1 0
7 6 5 4 3 2 1 0
8 7 6 5 4 3 2 1 0
9 8 7 6 5 4 3 2 1 0
```

Substitution

The shell performs substitution when it encounters an expression that contains one or more special characters.

Example

Here, the printing value of the variable is substituted by its value. Same time, "\n" is substituted by a new line –

```
#!/bin/sh  
  
a=10  
echo -e "Value of a is $a \n"
```

You will receive the following result. Here the **-e** option enables the interpretation of backslash escapes.

Value of a is 10

Following is the result without **-e** option –

Value of a is 10\n

The following escape sequences which can be used in echo command –

Sr.No.	Escape & Description
1	\\ backslash
2	\a alert (BEL)
3	\b backspace
4	\c suppress trailing newline
5	\f form feed
6	\n new line
7	\r

	carriage return
8	<code>\t</code> horizontal tab
9	<code>\v</code> vertical tab

You can use the **-E** option to disable the interpretation of the backslash escapes (default).

You can use the **-n** option to disable the insertion of a new line.

Command Substitution

Command substitution is the mechanism by which the shell performs a given set of commands and then substitutes their output in the place of the commands.

Syntax

The command substitution is performed when a command is given as –

``command``

When performing the command substitution make sure that you use the backquote, not the single quote character.

Example

Command substitution is generally used to assign the output of a command to a variable. Each of the following examples demonstrates the command substitution –

```
#!/bin/sh

DATE=`date`
echo "Date is $DATE"

USERS=`who | wc -l`
echo "Logged in user are $USERS"

UP=`date ; uptime`
echo "Uptime is $UP"
```

Upon execution, you will receive the following result –

```
Date is Thu Jul 2 03:59:57 MST 2009
Logged in user are 1
Uptime is Thu Jul 2 03:59:57 MST 2009
03:59:57 up 20 days, 14:03, 1 user, load avg: 0.13, 0.07, 0.15
```

Variable Substitution

Variable substitution enables the shell programmer to manipulate the value of a variable based on its state.

Here is the following table for all the possible substitutions –

Sr.No.	Form & Description
1	<code>\${var}</code> Substitute the value of <i>var</i> .
2	<code>\${var:-word}</code> If <i>var</i> is null or unset, <i>word</i> is substituted for var . The value of <i>var</i> does not change.
3	<code>\${var:=word}</code> If <i>var</i> is null or unset, <i>var</i> is set to the value of word .
4	<code>\${var:?message}</code> If <i>var</i> is null or unset, <i>message</i> is printed to standard error. This checks that variables are set correctly.
5	<code>\${var:+word}</code> If <i>var</i> is set, <i>word</i> is substituted for <i>var</i> . The value of <i>var</i> does not change.

Example

Following is the example to show various states of the above substitution –

```
#!/bin/sh

echo ${var:-"Variable is not set"}
echo "1 - Value of var is ${var}"

echo ${var:="Variable is not set"}
echo "2 - Value of var is ${var}"

unset var
echo ${var:+ "This is default value"}
echo "3 - Value of var is $var"

var="Prefix"
echo ${var:+ "This is default value"}
echo "4 - Value of var is $var"

echo ${var:? "Print this message"}
echo "5 - Value of var is ${var}"
```

Upon execution, you will receive the following result –

```
Variable is not set
1 - Value of var is
Variable is not set
2 - Value of var is Variable is not set
```


- 3 - Value of var is
This is default value
- 4 - Value of var is Prefix
Prefix
- 5 - Value of var is Prefix

The Metacharacters

Unix Shell provides various metacharacters which have special meaning while using them in any Shell Script and causes termination of a word unless quoted.

For example, `?` matches with a single character while listing files in a directory and an `*` matches more than one character. Here is a list of most of the shell special characters (also called metacharacters) –

`* ? [ ] ' " \ $ ; & ( ) | ^ < >` new-line space tab

A character may be quoted (i.e., made to stand for itself) by preceding it with a `\`.

Example

Following example shows how to print a `*` or a `?` –

```
#!/bin/sh
echo Hello; Word
```

Upon execution, you will receive the following result –

```
Hello
./test.sh: line 2: Word: command not found
```

shell returned 127

Let us now try using a quoted character –

```
#!/bin/sh
echo Hello\; Word
```

Upon execution, you will receive the following result –

```
Hello; Word
```

The `$` sign is one of the metacharacters, so it must be quoted to avoid special handling by the shell –

```
#!/bin/sh
echo "I have \$1200"
```

Upon execution, you will receive the following result –

```
I have $1200
```

The following table lists the four forms of quoting –

Sr.No.	Quoting & Description
1	Single quote All special characters between these quotes lose their special meaning.
2	Double quote Most special characters between these quotes lose their special meaning with these exceptions – • \$ • ` • \ • \ • \" • \\
3	Backslash Any character immediately following the backslash loses its special meaning.
4	Back quote Anything in between back quotes would be treated as a command and would be executed.

The Single Quotes

Consider an echo command that contains many special shell characters –

```
echo <-$1500.**>; (update?) [y|n]
```

Putting a backslash in front of each special character is tedious and makes the line difficult to read –

```
echo \<-\$1500.\*\>\; \ (update\?) \ [y|n]
```

There is an easy way to quote a large group of characters. Put a single quote (') at the beginning and at the end of the string –

```
echo '<-$1500.**>; (update?) [y|n]'
```

Characters within single quotes are quoted just as if a backslash is in front of each character. With this, the echo command displays in a proper way.

If a single quote appears within a string to be output, you should not put the whole string within single quotes instead you should precede that using a backslash (\) as follows –

```
echo 'It's Shell Programming'
```

The Double Quotes

Try to execute the following shell script. This shell script makes use of single quote –

```
VAR=ZARA
```

```
echo '$VAR owes <-$1500.**>; [ as of (`date +%m/%d`) ]'
```

Upon execution, you will receive the following result –

```
$VAR owes <-$1500.**>; [ as of (`date +%m/%d`) ]
```

This is not what had to be displayed. It is obvious that single quotes prevent variable substitution. If you want to substitute variable values and to make inverted commas work as expected, then you would need to put your commands in double quotes as follows –

```
VAR=ZARA  
echo "$VAR owes <-$1500.**>; [ as of (`date +%m/%d`) ]"
```

Upon execution, you will receive the following result –

```
ZARA owes <-$1500.**>; [ as of (07/02) ]
```

Double quotes take away the special meaning of all characters except the following –

- **\$** for parameter substitution
- Backquotes for command substitution
- **\\$** to enable literal dollar signs
- **\`** to enable literal backquotes
- **\"** to enable embedded double quotes
- **** to enable embedded backslashes
- All other **** characters are literal (not special)

Characters within single quotes are quoted just as if a backslash is in front of each character. This helps the echo command display properly.

If a single quote appears within a string to be output, you should not put the whole string within single quotes instead you should precede that using a backslash (****) as follows –

```
echo 'It's Shell Programming'
```

The Backquotes

Putting any Shell command in between **backquotes** executes the command.

Syntax

Here is the simple syntax to put any Shell **command** in between backquotes –

```
var=`command`
```

Example

The **date** command is executed in the following example and the produced result is stored in DATA variable.

```
DATE=`date`  
  
echo "Current Date: $DATE"
```

Upon execution, you will receive the following result –

```
Current Date: Thu Jul 2 05:28:45 MST 2009
```

Shell Input/Output Redirections

Output Redirection

The output from a command normally intended for standard output can be easily diverted to a file instead. This capability is known as output redirection.

If the notation `> file` is appended to any command that normally writes its output to standard output, the output of that command will be written to file instead of your terminal.

Check the following **who** command which redirects the complete output of the command in the users file.

```
$ who > users
```

Notice that no output appears at the terminal. This is because the output has been redirected from the default standard output device (the terminal) into the specified file. You can check the users file for the complete content –

```
$ cat users
oko    tty01  Sep 12 07:30
ai     tty15  Sep 12 13:32
ruth   tty21  Sep 12 10:10
pat    tty24  Sep 12 13:07
steve  tty25  Sep 12 13:03
$
```

If a command has its output redirected to a file and the file already contains some data, that data will be lost. Consider the following example –

```
$ echo line 1 > users
$ cat users
line 1
$
```

You can use `>>` operator to append the output in an existing file as follows –

```
$ echo line 2 >> users
$ cat users
line 1
line 2
$
```

Input Redirection

Just as the output of a command can be redirected to a file, so can the input of a command be redirected from a file. As the **greater-than character** `>` is used for output redirection, the **less-than character** `<` is used to redirect the input of a command.

The commands that normally take their input from the standard input can have their input redirected from a file in this manner. For example, to count the number of lines in the file *users* generated above, you can execute the command as follows –

```
$ wc -l users
2 users
$
```

Upon execution, you will receive the following output. You can count the number of lines in the file by redirecting the standard input of the **wc** command from the file *users* –

```
$ wc -l < users
2
```

\$

Note that there is a difference in the output produced by the two forms of the `wc` command. In the first case, the name of the file `users` is listed with the line count; in the second case, it is not.

In the first case, `wc` knows that it is reading its input from the file `users`. In the second case, it only knows that it is reading its input from standard input so it does not display file name.

Here Document

A **here document** is used to redirect input into an interactive shell script or program.

We can run an interactive program within a shell script without user action by supplying the required input for the interactive program, or interactive shell script.

The general form for a **here** document is –

```
command << delimiter
document
delimiter
```

Here the shell interprets the `<<` operator as an instruction to read input until it finds a line containing the specified delimiter. All the input lines up to the line containing the delimiter are then fed into the standard input of the command.

The delimiter tells the shell that the **here** document has completed. Without it, the shell continues to read the input forever. The delimiter must be a single word that does not contain spaces or tabs.

Following is the input to the command **`wc -l`** to count the total number of lines –

```
$wc -l << EOF
  This is a simple lookup program
    for good (and bad) restaurants
    in Cape Town.
EOF
3
$
```

You can use the **here document** to print multiple lines using your script as follows –

[Live Demo](#)

```
#!/bin/sh

cat << EOF
This is a simple lookup program
for good (and bad) restaurants
in Cape Town.
EOF
```

Upon execution, you will receive the following result –

```
This is a simple lookup program
for good (and bad) restaurants
in Cape Town.
```

The following script runs a session with the **`vi`** text editor and saves the input in the file **`test.txt`**.

```
#!/bin/sh

filename=test.txt
vi $filename <<EndOfCommands
i
This file was created automatically from
a shell script
```

```
^[
ZZ
EndOfCommands
```

If you run this script with vim acting as vi, then you will likely see output like the following –

```
$ sh test.sh
Vim: Warning: Input is not from a terminal
$
```

After running the script, you should see the following added to the file **test.txt** –

```
$ cat test.txt
This file was created automatically from
a shell script
```